

Week 3 Office Hour Notes

3.0 Errata

9/16/24 6pm. These notes will be updated with homework solutions sometime around Tuesday 9/17/24.

3.1 Omitting Parentheses

The rules for omitting parentheses are as follows (from page 33 of Enderton)

Omitting Parentheses

Hereafter when naming wffs, we will not feel compelled to mention explicitly every parenthesis. To establish a more compact notation, we now adopt the following conventions:

1. The outermost parentheses need not be explicitly mentioned. For example, when we write " $\mathbf{A \wedge B}$ " we are referring to $(\mathbf{A \wedge B})$.
2. The negation symbol applies to as little as possible. For example, $\neg \mathbf{A \wedge B}$ is $(\neg \mathbf{A}) \wedge \mathbf{B}$, i.e., $((\neg \mathbf{A}) \wedge \mathbf{B})$. It is not the same as $(\neg (\mathbf{A \wedge B}))$.
3. The conjunction and disjunction symbols apply to as little as possible, given that convention 2 is to be observed. For example,

$$\mathbf{A \wedge B \rightarrow \neg C \vee D} \text{ is } ((\mathbf{A \wedge B}) \rightarrow ((\neg \mathbf{C}) \vee \mathbf{D})).$$

4. Where one connective symbol is used repeatedly, grouping is to the right:

$$\alpha \wedge \beta \wedge \gamma \text{ is } \alpha \wedge (\beta \wedge \gamma).$$

$$\alpha \rightarrow \beta \rightarrow \gamma \text{ is } \alpha \rightarrow (\beta \rightarrow \gamma).$$

It must be admitted that these conventions violate what was said earlier about naming expressions. We can get away with this only because we no longer have any interest in naming expressions that are not wffs.

%GIVE NEW IDEA FOR REFRAMING RULES IN TERMS OF ORDER OF OPERATION

Here are a few examples:

1. $(A \vee (\neg A))$

First, we can omit the outter parentheses, giving us

$$A \vee (\neg A).$$

Next, we note that $\neg$ without parentheses covers as little as possible by rule 2, so we can eliminate the set of parentheses around the negation phrase $\neg A$, and we're left with

$$A \vee \neg A.$$

$$2. ((A_1 \vee A_2) \wedge A_3)$$

First, we can omit the outer parentheses, giving us

$$(A_1 \vee A_2) \wedge A_3.$$

Next, we note that $\vee, \wedge$ applies to as little as possible, however we need the parentheses in this case so that we can determine the order the operation is applied.

$$3. ((A \rightarrow B) \leftrightarrow ((\neg B) \rightarrow (\neg A)))$$

First, we can omit the outer parentheses, giving us

$$(A \rightarrow B) \leftrightarrow ((\neg B) \rightarrow (\neg A)).$$

We can then omit the parentheses around the negation symbols since we know by rule 2 that this symbol apply to as little as possible, giving us

$$(A \rightarrow B) \leftrightarrow (\neg B \rightarrow \neg A).$$

However, since there's no order of operations giving $\rightarrow$ any priority over $\leftrightarrow$, and for that matter vice versa, we must keep the parentheses we currently have.

$$4. ((A \rightarrow B) \rightarrow (\neg B \rightarrow \neg A))$$

First, as in the previous example, we can omit the outer parentheses as well as the ones around the negation symbols, giving us

$$(A \rightarrow B) \rightarrow (\neg B \rightarrow \neg A).$$

But since we have $\rightarrow$ in the middle instead of $\leftrightarrow$, we can use rule 4 of repetition of symbols

and omit the parentheses to the right to give us

$$(A \rightarrow B) \rightarrow \neg B \rightarrow \neg A,$$

and from there we are done.

3.2 Regular vs. Polish Notation

Recall how polish notation is constructed (from page 32 of Enderton):

Polish Notation

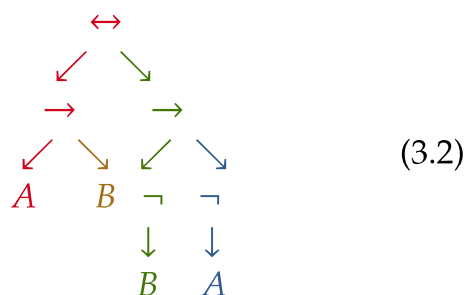
It is possible to avoid both ambiguity and parentheses. This can be done by a very simple device. Instead of, for example, $(\alpha \wedge \beta)$ we use $\wedge\alpha\beta$. Let the set of P-wffs be the set generated from the sentence symbols by the five operations

$$\begin{aligned} \mathcal{D}_{\neg}(\alpha) &= \neg\alpha, & \mathcal{D}_{\forall}(\alpha, \beta) &= \forall\alpha\beta, \\ \mathcal{D}_{\wedge}(\alpha, \beta) &= \wedge\alpha\beta, & \mathcal{D}_{\rightarrow}(\alpha, \beta) &= \rightarrow\alpha\beta, \\ \mathcal{D}_{\leftrightarrow}(\alpha, \beta) &= \leftrightarrow\alpha\beta. \end{aligned}$$

Note that Polish Notation can be written pretty easily from the construction tree from going left to right from the branches of the tree and writing the symbols down accordingly. For example, take the wff

$$((A \rightarrow B) \leftrightarrow ((\neg B) \rightarrow (\neg A))) \quad (3.1),$$

which has the construction tree



We start our expression in polish notation with the leftmost branch (in red) by writing

$$\leftrightarrow \rightarrow A$$

Then we continue with our next branch (in orange) by writing

$$\leftrightarrow \rightarrow AB,$$

then next (in green) with

$$\leftrightarrow \rightarrow AB \rightarrow \neg B,$$

then finally (in blue) with

$$\leftrightarrow \rightarrow AB \rightarrow \neg B \neg A,$$

and so expression (3.1) has polish notation

$$\leftrightarrow \rightarrow AB \rightarrow \neg B \neg A. \quad (3.3)$$

Regular Notation to Polish Notation

This can be done by using the Parsing Algorithm to get from regular notation to a construction tree, and then using the construction tree to, as demonstrated with using the construction tree in (3.2) to write it in polish notation (as done in (3.3)).

If you don't recall, the Parsing algorithm is done as follows (as explained in page 31 of Enderton):

The algorithm consists of the following four steps:

1. If all minimal vertices (the ones at the bottom) have sentence symbols, then the procedure is completed. (The given expression is indeed a wff, and we have constructed its tree.) Otherwise, select a minimal vertex having an expression that is not a sentence symbol. We examine that expression.

2. The first symbol must* be (. If the second symbol is the negation symbol, jump to step 4. Otherwise, go on to step 3.

3. Scan the expression from the left until first reaching $(\alpha$, where α is a nonempty expression having a balance between left and right parentheses.** Then α is the first of the two constituents. The next symbol must* be $\wedge$, $\vee$, $\rightarrow$, or $\leftrightarrow$. This is the principal connective. The remainder of the expression, β , must* consist of an expression β and a right parenthesis. We extend the tree by creating two new vertices below the present one, with α as the expression at the “left child” vertex, and β as the expression at the “right child” vertex. Return to step 1.

4. The first two symbols are now known to be $(\neg$. The remainder of the expression, β , must* consist of an expression β and a right parenthesis. We extend the tree by creating one new vertex below the present one, with β as the expression at the child vertex. Return to step 1.

As a demonstration of the Parsing Algorithm, we shall show the algorithm being done on expression (3.1) to derive the construction tree in (3.2) as so:

We employ step 1 on the original expression (3.1) and call it α_1 , i.e.

$((A \rightarrow B) \leftrightarrow ((\neg B) \rightarrow (\neg A)))$, and look at it as the single vertex tree.

α_1 , (3.4)

then we employ step 2 on α_1 and realize it is of the form $(\alpha_{2,1} \leftrightarrow \alpha_{2,2})$, where

$\alpha_{2,1}$ is $(A \rightarrow B)$,

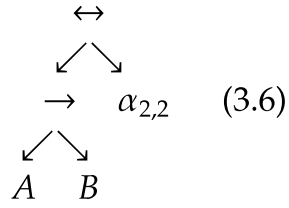
$\alpha_{2,2}$ is $((\neg B) \rightarrow (\neg A))$,

go into step 3, extend tree in (3.4) to

$$\begin{array}{c}
 \leftrightarrow \\
 \swarrow \quad \searrow \\
 \alpha_{2,1} \quad \alpha_{2,2}
 \end{array}
 \quad (3.5)$$

Next, we go back to step 1, select $\alpha_{2,1}$, employ step 2, and find the derivative expressions

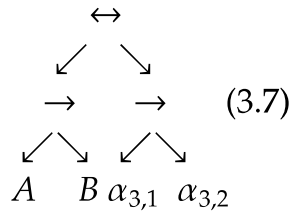
A, B connected by $\rightarrow$, then go to step 3 and extend the tree in (3.5) to



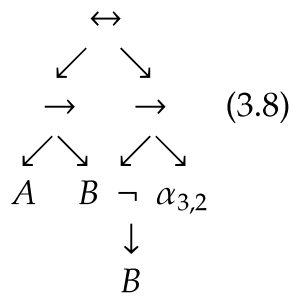
Next, we go back again to step 1, select $\alpha_{2,1}$, employ step 2, and find the derivative expressions $\alpha_{3,1}, \alpha_{3,2}$, connected by $\rightarrow$, where

$\alpha_{3,1}$ is $(\neg B)$,
 $\alpha_{3,2}$ is $(\neg A)$,

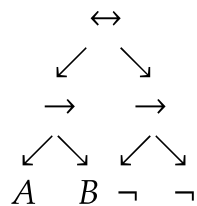
go into step 3, and extend the tree in (3.6) to



From there, we go back to step 1, select $\alpha_{3,1}$, employ step 2, and find the derivative expression B connected by $\neg$, go into step 4, and extend the tree in (3.7) to



Then we similarly employ step 1, step 2, and step 4 on $\alpha_{3,2}$, which extends the tree in (3.8) to



$\downarrow \quad \downarrow$
 $B \quad A$

Finally, since all the minimal vertices are sentence symbols, we are done.

Polish Notation to Regular Notation

In the below discussion forum post, I explain how one can translate from polish notation to regular notation, which entails building the tree from the polish expression in the way I describe, and from the construction tree, we can move down from the tree and construct the expression in regular notation.



Sep 12 12:21pm

To elaborate on that, I think building the construction tree from bottom to top deciphering the polish notation from right to left is the way to go, in contrast to building the construction tree from regular formula notation using the parsing algorithm, which I'm pretty sure is a top to bottom construction.

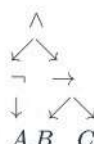
Let's take the example $((\neg A) \wedge (B \rightarrow C))$, which has polish notation $\wedge \neg A \rightarrow BC$. Using the parsing algorithm, we start at the $\wedge$ symbol and create two branches as so..



Then we look at each branch and realize the first branch is led by the negation symbol and the second one is led by the implication symbol



Then we go further down into the branch and realize that the sentence symbols below that we finish the construction tree at



One can definitely use a top to bottom parsing algorithm for polish notation (as the powerpoint seems to do), but personally I find it easier to go from bottom to top, i.e., with the example $\wedge \neg A \rightarrow BC$ start with the sentence symbols B, C at the furthest right, and then notice the connective $\rightarrow$, and then you have



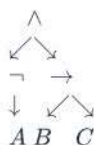
After that, you have sentence symbol A , which we know is a root vertex, but we're figuring out how to connect it, and right next to A is the $\neg$ connective, which we know connects to A , leaving us with



And finally, we have $\wedge$, which we can see connects $\neg A$ aka $\neg$ with $B \rightarrow C$, aka $\rightarrow$, to give us the final construction



tree



Anybody want to take a crack at finding the construction tree, and then the original (omitting) parentheses notation with the polish notation expression $\leftrightarrow \wedge A \vee BC \vee \wedge AB \wedge AC$?

As I state in this post, one can alternatively use the parsing algorithm to construct the tree, but student Evalin Lopez reached out to me and told me that she likes to go from polish

notation directly to the expression in a top-to-bottom style similar to the parsing algorithm by going through the polish notation expression left to right and applying repeated substitutions until no substitutions need to be made.

So with the example $\neg A \rightarrow BC$, we start by writing our final expression as

$$\phi_{1,1} \wedge \phi_{1,2}$$

and then we try to find $\phi_{1,1}$ by starting with the negation symbol $\neg$, then applying substitution to $\phi_{1,1}$ to get $\neg\phi_{2,1}$, hence our final expression derives to

$$\neg\phi_{2,1} \wedge \phi_{1,2}$$

dig further and realize that $\phi_{2,1}$ is A , hence $\phi_{1,1}$ is $\neg A$, and further break down the final expression to

$$\neg A \wedge \phi_{1,2}.$$

Note that we are left with $\rightarrow BC$ for $\phi_{1,2}$, where we repeat the process and break it down to $\phi_{3,1} \rightarrow \phi_{3,2}$, giving us

$$\neg A \wedge (\phi_{3,1} \rightarrow \phi_{3,2}),$$

as our final expression. We then substitute B for $\phi_{3,1}$, which leaves us with

$$\neg A \wedge (B \rightarrow \phi_{3,2}),$$

and finally substitute C for $\phi_{3,2}$, and we finish with the desired expression

$$\neg A \wedge (B \rightarrow C).$$

finish the final expression.